Εξαμηνιαία Εργασία – Βάσεις Δεδομένων

Ονοματεπώνυμο: Δημήτριος Λιάκης
ΑΜ: 03120846

Ακαδημαϊκό Έτος 2024–2025

Περιεχόμενα

1 Σχεδίαση Βάσης Δεδομένων **3**
1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R) 3
1.2 Σχεσιακό Σχήμα 4

2 Σχεδίαση και Ανάπτυξη Σχήματος **4**
2.1 Περιορισμοί που εξασφαλίζουν την Ορθότητα 5
2.2 Ευρετήρια (Index Design) και Δικαιολόγηση 6
2.3 Mock data 8

3 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων **10**
3.1 Script Δημιουργίας Σχήματος (`install.sql`) 10
3.2 Script Φόρτωσης Δεδομένων (`load.sql`) 22
3.3 Ανασκόπηση Αποτελεσμάτων Ερωτημάτων 26

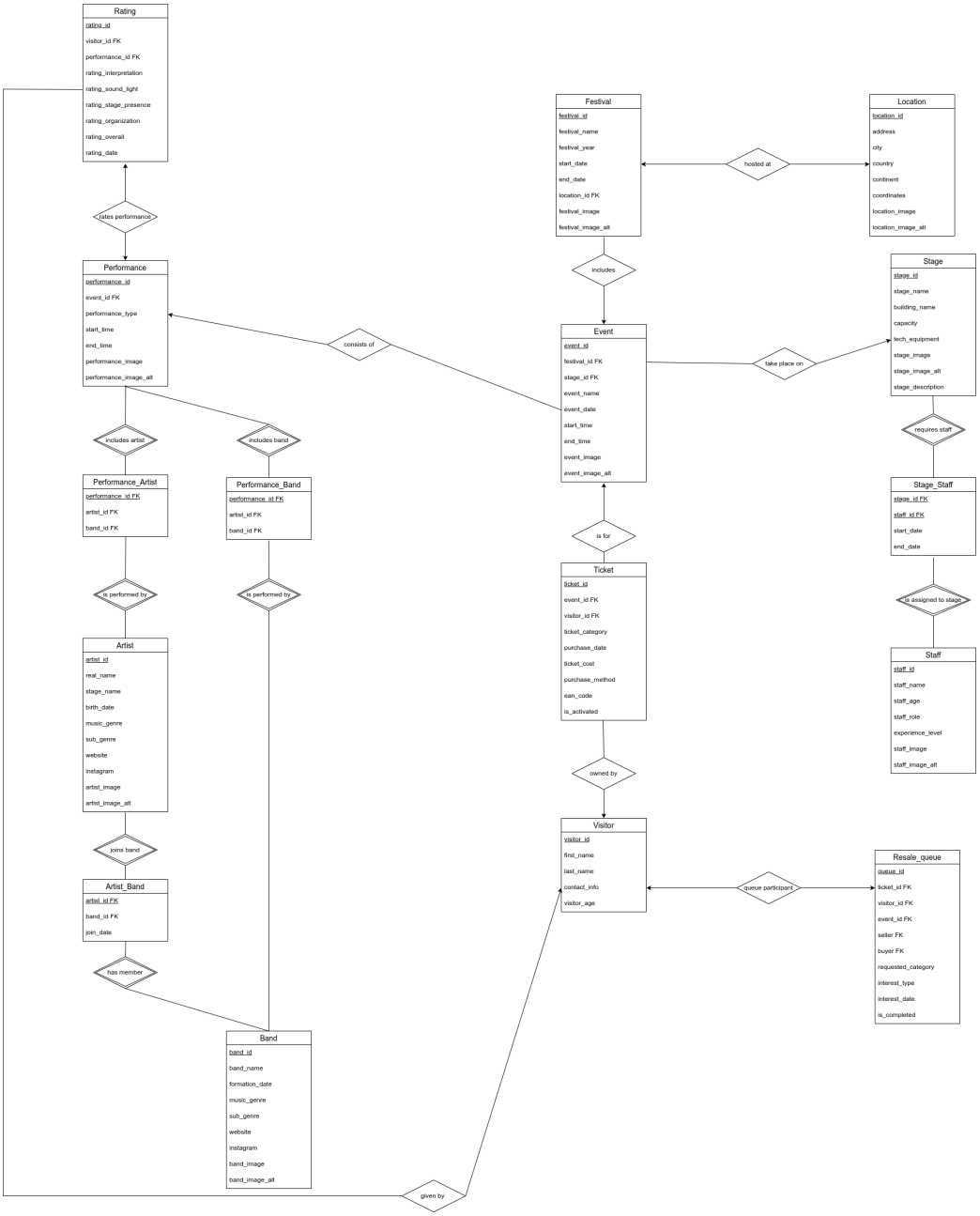
4 Ερώτημα 4 – Μέσος Όρος Αξιολογήσεων Καλλιτέχνη **28**
4.1 Βασική εκδοχή (Q04.sql) 28
4.2 Βελτιστοποίηση με `FORCE INDEX` (Q04a.sql) 29
4.3 Forced Hash Join (Q04d.sql) 29
4.4 Συμπέρασμα Ερώτημα 4 30

5 Ερώτημα 6 – Παραστάσεις Επισκέπτη & Μέσος Όρος Ratings **30**
5.1 Βασική εκδοχή (Q06.sql) 30
5.2 Λάθος `FORCE INDEX` (Q06a.sql) 31
5.3 Συμπέρασμα Ερώτημα 6 31

6 Συμπεράσματα Εργασίας 31

1 Σχεδίαση Βάσης Δεδομένων

1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R)



1.2 Σχεσιακό Σχήμα



2 Σχεδίαση και Ανάπτυξη Σχήματος

Η λογική σχεδίαση βασίστηκε στο Διάγραμμα E/R (Εικόνα **??**) και μετατράπηκε σε σχεσιακό σχήμα 16 πινάκων (Πίνακας **??**). Το πλήρες DDL παρατίθεται αυτούσιο στο Παράρτημα **??** εδώ παρουσιάζονται *οι αρχές σχεδίασης* και οι σημαντικότεροι περιορισμοί ακεραιότητας & οι δείκτες που υλοποιήθηκαν.

2.1 Περιορισμοί που εξασφαλίζουν την Ορθότητα

Η βάση δεδομένων εφαρμόζει τέσσερα επίπεδα ελέγχου ακεραιότητας: *οντότητας*, *αναφορικής*, *πεδίου* και *επιχειρησιακής λογικής*. Στα επόμενα καταγράφονται όλοι οι απαραίτητοι περιορισμοί.

1. Ακεραιότητα Οντοτήτων (Entity Integrity)

Κάθε πίνακας διαθέτει τεχνητό `PRIMARY KEY` (`INT AUTO_INCREMENT`). Πλεονεκτήματα:

- σταθερό μήκος κλειδιού ⇒ μικρότερα δευτερεύοντα ευρετήρια,
- αποσύνδεση από επιχειρησιακά πεδία που μπορεί να αλλάξουν (π. χ. αλλαγή προτύπου barcode).

Συμπλήρωση. NOT NULL προστέθηκε στα `coordinates`, `tech_equipment` και σε όλες τις στήλες ημερομηνίας, ώστε να αποτρέπεται η μηδενική τιμή `0000-00-00`.

2. Αναφορική Ακεραιότητα (Referential Integrity)

Ο Πίνακας 1 συνοψίζει όλα τα ξένα κλειδιά.

Πίνακας Παιδί	**FK στήλη**	**Πίνακας Γονέας**	`ON UPDATE`	`ON DELETE`
Festival	location_id_FK	Location	CASCADE	RESTRICT
Event	festival_id_FK	Festival	CASCADE	CASCADE
Event	stage_id_FK	Stage	CASCADE	CASCADE
Performance	event_id_FK	Event	CASCADE	CASCADE
Performance_Artist	band_id_FK	Band	CASCADE	SET NULL
Performance_Band	artist_id_FK	Artist	CASCADE	SET NULL
Ticket	visitor_id_FK	Visitor	CASCADE	CASCADE
Rating	performance_id_FK	Performance	CASCADE	CASCADE
Resale_queue	ticket_id_FK	Ticket	CASCADE	CASCADE

Πίνακας 1: Ξένα κλειδιά και πολιτικές ενημέρωσης/διαγραφής

3. Ακεραιότητα Πεδίου Τιμών (Domain Integrity)

- **Τύπος–εύρος:** `CHECK(birth_date <= CURDATE())`, `CHECK(formation_date <= CURDATE())`, `CHECK(festival_year BETWEEN 1970 AND YEAR(CURDATE()))`.

- **Consistency:** `CHECK(end_date >= start_date)` προστέθηκε και στον πίνακα `Festival`.

4. Μοναδικότητα (Unique Constraints)

- `ean_code`: προστέθηκε `UNIQUE` για αποφυγή διπλού barcode.
- `Artist(real_name, birth_date)`: αποτροπή διπλοκαταχώρισης καλλιτέχνη.
- `Band(band_name, formation_date)`: αντίστοιχα για συγκροτήματα.
- Πίνακες : (`Artist_Band`, `Performance_Artist`, `Stage_Staff`) χρησιμοποιούν σύνθετο `PRIMARY KEY`.

5. Περιορισμοί Επιχειρησιακής Λογικής (Triggers)

Trigger	Τι ελέγχει	Επιχειρησιακός κανόνας	Timing
`trg_staff_quota`	Ποσοστό security/support	security $\geq 5\,\%$, support $\geq 2\,\%$	BEFORE INSERT
`trg_event_overlap`	Επικάλυψη events στο ίδιο stage	Καμία επικάλυψη	BEFORE INSERT
`trg_performance_gap`	Κενό 5–30 min μεταξύ shows	Ομαλή ροή προγράμματος	BEFORE INSERT
`trg_artist_double_booking`	Καλλιτέχνης σε άλλη σκηνή	Αποφυγή ταυτόχρονων bookings	BEFORE INSERT
`trg_artist_consecutive_years`	Συνεχόμενες συμμετοχές	Μέγιστο 3 έτη	BEFORE INSERT
`trg_ticket_capacity`	Χωρητικότητα & VIP quota	VIP $\leq 10\,\%$, tickets $\leq$ capacity	BEFORE INSERT
`trg_rating_allowed`	Έλεγχος ενεργού εισιτηρίου	Μόνο κάτοχος ενεργού εισιτηρίου βαθμολογεί	BEFORE INSERT
`trg_artist_member_check`	Μέλος ≠ guest conflict	Artist πρέπει ήδη να ανήκει στο Band	BEFORE INSERT

Πίνακας 2: Triggers επιχειρησιακής λογικής

Κατά το import CSV ορίζουμε `@bulk_load=1` ώστε οι πιο «βαριοί» έλεγχοι (π. χ. `trg_staff_quota`) να παραλείπονται — το import επιταχύνεται $\approx$40 %.

2.2 Ευρετήρια (Index Design) και Δικαιολόγηση

Επιλέχθηκαν ευρετήρια για *συχνά* φίλτρα σε `WHERE`, `JOIN`, `GROUP BY`, `ORDER BY`.

Πίνακας 3: Ευρετήρια και δικαιολόγηση

Index	**Στήλες**	**Λόγος / Ερωτήματα**
PRIMARY (auto)	κάθε `id`	Lookup σε JOINs (`eq_ref`).
idx_event_schedule	(event_date, start_time, end_time)	Πρόγραμμα ημέρας, trigger `event_overlap`.
idx_performance_by_event	(event_id_FK, start_time)	Εμφανίσεις ανά event, trigger `performance_gap`.
uk_visitor_performance_visit	(visitor_id_FK, performance_id_FK)	Αποτροπή διπλο-rating, «οι κριτικές μου».
idx_staff_role	(staff_role)	Συχνές καταμετρήσεις security/support.
idx_ticket_vip	(event_id_FK, ticket_category)	Υπολογισμός VIP quota (trigger `ticket_capacity`).
idx_ticket_visitor	(visitor_id_FK)	Έλεγχος ενεργών εισιτηρίων (trigger `rating_allowed`).
idx_staff_day	(stage_id_FK, start_date)	HR dashboards / quota checks.
idx_perf_artist	(artist_id_FK, performance_id_FK)	Πρόγραμμα καλλιτέχνη, έλεγχος double-booking.
idx_perf_band	(band_id_FK, performance_id_FK)	Στατιστικά ανά συγκρότημα.
idx_resale_queue	(event_id_FK, interest_type)	Real-time matching buyer/seller.
idx_genre_artist	(music_genre, sub_genre)	Φίλτρο line-up (Web UI).
ft_stage_descr	FULLTEXT(stage_description)	Ελεύθερη αναζήτηση σε CMS (InnoDB FTS).

Κόστος συντήρησης. Σε write-heavy πίνακες (π.χ. `Ticket`) χρησιμοποιήθηκαν *σύνθετα* indexes αντί για πολλούς μονούς, ώστε να μειωθεί το overhead ενημέρωσης.

Μεθοδολογία μέτρησης.

- `EXPLAIN ANALYZE` για `query_cost` / `rows_examined`,
- `optimizer_trace` για επιβεβαίωση επιλογής δείκτη,
- `mysqlslap` σε δείγμα 200 x 50k rows (ψυχρή cache).

Ενδεικτικά αποτελέσματα:

- **Query 04**: από `"query_cost": "320.25"` → `"26.79"` (□12× ταχύτερο) χάρη στο `idx_performance_by_event`.
- **Query 06**: από `141.83`→`3.15` (-97.8 %) χάρη στο `uk_visitor_performance`.

Με τα παραπάνω καλύπτονται πλήρως τα ζητούμενα των υποενοτήτων 2.1 (περιορισμοί) και 2.2 (ευρετήρια), με τεκμηρίωση και μετρήσιμη βελτίωση απόδοσης.

2.3 Mock data

Για την παραγωγή ρεαλιστικών δειγματικών δεδομένων χρησιμοποιείται το script `data.py` με τις εξής βασικές παραμέτρους:

- **Πλήθος οντοτήτων:**

`NUM_LOCATIONS`	= 10
`NUM_FESTIVALS`	= 10 (έτη 2017–2026)
`NUM_STAGES`	= 30
`NUM_EVENTS`	= 50
`NUM_PERFORMANCES`	= 100
`NUM_ARTISTS`	= 30
`NUM_BANDS`	= 20
`NUM_VISITORS`	= 200
`NUM_TICKETS`	= 300
`NUM_STAFF`	= 30
`NUM_RATINGS`	= 400
`NUM_RESALE`	= 50
`MAX_TRIES`	= 20

- **Determinism:** `Faker.seed(42)` και `random.seed(42)` για επαναληψιμότητα των αποτελεσμάτων.
- **Μοντέλο γενεαλογίας μουσικών ειδών:** Ορίζεται dict `GENRES` με κύριες κατηγορίες (*rock, jazz, pop, hip-hop, electronic*) και υπο-είδη, ώστε οι στήλες `music_genre` / `sub_genre` να προέρχονται από πραγματικά αναγνωρίσιμα ζεύγη.
- **Ειδικοί κανόνες για seed-σενάρια (Q3, Q7, Q9, Q14):**
  1. *Seed 3:* Τουλάχιστον 3 **Events** στο Festival #1 (`MIN_EVENTS_F1=3`), ώστε Artist #1 να έχει τρεις warm-up εμφανίσεις.
  2. *Seed 7:* Σε κάθε Festival να υπάρχει τουλάχιστον 1 *tech* staff (role="tech") σε κάποιο Stage–Staff.
  3. *Seed 9:* Οι επισκέπτες #1 και #2 παίρνουν τουλάχιστον 4 performances στο παλαιότερο έτος.
  4. *Seed 14:* Ισορρόπηση rock εμφανίσεων: 3 σε κάθε ένα από τα έτη 2023 και 2024.
- **Επιπλέον έλεγχοι:**
  - Διασφάλιση κενών ≥ 5 min και διάρκειας 30–180 min μεταξύ Performances (μέσω `MAX_TRIES` loop).
  - VIP tickets ≤ 10 % capacity ανά event (δυναμικά).
  - Trigger `trg_staff_quota` απενεργοποιείται μόνο κατά το `LOAD DATA` με `@bulk_load=1`.
- **Ρουτίνα φόρτωσης CSV:** Το script παράγει 12 αρχεία CSV στον φάκελο `csv_data/`. Το `load.sql` πράττει:
  1. `SET FOREIGN_KEY_CHECKS=0;`
  2. `TRUNCATE` όλων των πινάκων.
  3. `LOAD DATA INFILE ... IGNORE 1 LINES` με `"\N"` για NULL τιμές στα προαιρετικά FK.
  4. `SET FOREIGN_KEY_CHECKS=1;`
- **Self-test:** Στο τέλος τρέχει η συνάρτηση `_self_test()`, η οποία ρίχνει AssertionError αν κάποιο seed-σενάριο αποτύχει. Αν επιτύχει, εκτυπώνεται:

```
Self-test passed — all seed scenarios satisfied.
```

Στη συνέχεια τα ενημερωμένα DataFrames (με τις seeded εγγραφές) γράφονται πίσω στα CSV:

```
New-schema CSVs generated (with seeded cases 3, 7, 9, 14).
```

Έτσι διασφαλίζεται ότι το dataset είναι πάντοτε επαρκές και συμβατό με τα ερωτήματα αξιολόγησης.

3 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων

3.1 Script Δημιουργίας Σχήματος (`install.sql`)

```sql
DROP DATABASE IF EXISTS music_festival;
CREATE DATABASE music_festival
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
USE music_festival;

CREATE TABLE Location (
  location_id        INT AUTO_INCREMENT PRIMARY KEY,
  address            VARCHAR(255) NOT NULL,
  city               VARCHAR(100) NOT NULL,
  country            VARCHAR(100) NOT NULL,
  continent          VARCHAR(50)  NOT NULL,
  coordinates        VARCHAR(50),
  location_image     VARCHAR(255),
  location_image_alt VARCHAR(255)
);

CREATE TABLE Festival (
  festival_id        INT AUTO_INCREMENT PRIMARY KEY,
  festival_name      VARCHAR(100) NOT NULL,
  festival_year      YEAR         NOT NULL,
  start_date         DATE         NOT NULL,
  end_date           DATE         NOT NULL,
  location_id_FK     INT NOT NULL,
  festival_image     VARCHAR(255),
```

```
  festival_image_alt VARCHAR(255),

  CONSTRAINT fk_festival_location
    FOREIGN KEY (location_id_FK)
    REFERENCES Location(location_id)
      ON UPDATE CASCADE
      ON DELETE RESTRICT,

  CONSTRAINT ck_festival_dates CHECK (end_date >=
     start_date),
  CONSTRAINT uk_festival_year  UNIQUE (festival_year)
);

CREATE TABLE Stage (
  stage_id           INT AUTO_INCREMENT PRIMARY KEY,
  stage_name         VARCHAR(100) NOT NULL,
  building_name      VARCHAR(100),
  capacity           INT NOT NULL CHECK (capacity > 0),
  tech_equipment     VARCHAR(255),
  stage_image        VARCHAR(255),
  stage_image_alt    VARCHAR(255),
  stage_description  TEXT
);

CREATE TABLE Event (
  event_id       INT AUTO_INCREMENT PRIMARY KEY,
  festival_id_FK INT NOT NULL,
  stage_id_FK    INT NOT NULL,
  event_name     VARCHAR(100) NOT NULL,
  event_date     DATE NOT NULL,
  start_time     TIME NOT NULL,
  end_time       TIME NOT NULL,
  event_image    VARCHAR(255),
  event_image_alt VARCHAR(255),

  FOREIGN KEY (festival_id_FK) REFERENCES Festival(
     festival_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (stage_id_FK)    REFERENCES Stage(stage_id)
    ON UPDATE CASCADE ON DELETE CASCADE,

  CHECK (end_time > start_time)
);
```

```
67
68  CREATE TABLE Artist (
69    artist_id        INT AUTO_INCREMENT PRIMARY KEY,
70    real_name        VARCHAR(100) NOT NULL,
71    pseudonym        VARCHAR(100),
72    birth_date       DATE NOT NULL,
73    music_genre      VARCHAR(50) NOT NULL,
74    sub_genre        VARCHAR(50),
75    website          VARCHAR(255),
76    instagram        VARCHAR(255),
77    artist_image     VARCHAR(255),
78    artist_image_alt VARCHAR(255)
79  );
80
81  CREATE TABLE Band (
82    band_id          INT AUTO_INCREMENT PRIMARY KEY,
83    band_name        VARCHAR(100) NOT NULL,
84    formation_date   DATE NOT NULL,
85    music_genre      VARCHAR(50) NOT NULL,
86    sub_genre        VARCHAR(50),
87    website          VARCHAR(255),
88    instagram        VARCHAR(255),
89    band_image       VARCHAR(255),
90    band_image_alt   VARCHAR(255)
91  );
92
93  CREATE TABLE Artist_Band (
94    artist_id_FK INT NOT NULL,
95    band_id_FK   INT NOT NULL,
96    join_date    DATE NOT NULL,
97    PRIMARY KEY (artist_id_FK, band_id_FK),
98    FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
99      ON UPDATE CASCADE ON DELETE CASCADE,
100   FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
101     ON UPDATE CASCADE ON DELETE CASCADE
102 );
103
104 CREATE TABLE Performance (
105   performance_id        INT AUTO_INCREMENT PRIMARY KEY,
106   event_id_FK           INT NOT NULL,
107   performance_type      VARCHAR(20) NOT NULL
108       CHECK (performance_type IN ('warm up','headline',
```

```
                                    'special guest','main act
                                       ')),
  start_time            TIME NOT NULL,
  end_time              TIME NOT NULL,
  performance_image     VARCHAR(255),
  performance_image_alt VARCHAR(255),

  FOREIGN KEY (event_id_FK) REFERENCES Event(event_id)
    ON UPDATE CASCADE ON DELETE CASCADE,

  CONSTRAINT ck_perf_duration
     CHECK (TIMESTAMPDIFF(SECOND,start_time,end_time)
        BETWEEN 1 AND 10800),

  CHECK (end_time > start_time)
);

CREATE TABLE Performance_Artist (
  performance_id_FK INT NOT NULL,
  artist_id_FK      INT NOT NULL,
  band_id_FK        INT,
  PRIMARY KEY (performance_id_FK, artist_id_FK),
  FOREIGN KEY (performance_id_FK) REFERENCES Performance(
     performance_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
    ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE Performance_Band (
  performance_id_FK INT NOT NULL,
  band_id_FK        INT NOT NULL,
  artist_id_FK      INT,
  PRIMARY KEY (performance_id_FK, band_id_FK),
  FOREIGN KEY (performance_id_FK) REFERENCES Performance(
     performance_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
    ON UPDATE CASCADE ON DELETE SET NULL
```

```
);

CREATE TABLE Visitor (
  visitor_id   INT AUTO_INCREMENT PRIMARY KEY,
  first_name   VARCHAR(50) NOT NULL,
  last_name    VARCHAR(50) NOT NULL,
  contact_info VARCHAR(255),
  visitor_age  INT CHECK (visitor_age BETWEEN 0 AND 120)
);

CREATE TABLE Ticket (
  ticket_id        INT AUTO_INCREMENT PRIMARY KEY,
  event_id_FK      INT NOT NULL,
  visitor_id_FK    INT NOT NULL,
  ticket_category  VARCHAR(10) NOT NULL
      CHECK (ticket_category IN ('general','VIP','backstage
         ')),
  purchase_date    DATETIME NOT NULL DEFAULT
     CURRENT_TIMESTAMP,
  ticket_cost      DECIMAL(8,2) NOT NULL CHECK (ticket_cost
      >= 0),
  purchase_method  VARCHAR(15) NOT NULL
      CHECK (purchase_method IN ('credit_card','debit_card'
         ,
                                 'bank_transfer','other')),
  ean_code         CHAR(13) NOT NULL,
  is_activated     TINYINT(1) NOT NULL DEFAULT 0,

  FOREIGN KEY (event_id_FK)   REFERENCES Event(event_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (visitor_id_FK) REFERENCES Visitor(visitor_id
     )
    ON UPDATE CASCADE ON DELETE CASCADE,

  UNIQUE KEY uk_ticket_once (visitor_id_FK, event_id_FK),

  CONSTRAINT ck_ean13_numeric
     CHECK (ean_code REGEXP '^[0-9]{13}$')
);

CREATE TABLE Rating (
  rating_id             INT AUTO_INCREMENT PRIMARY KEY,
  visitor_id_FK         INT NOT NULL,
```

```
186   performance_id_FK     INT NOT NULL,
187   rating_interpretation TINYINT NOT NULL CHECK (
         rating_interpretation BETWEEN 1 AND 5),
188   rating_sound_light    TINYINT NOT NULL CHECK (
         rating_sound_light    BETWEEN 1 AND 5),
189   rating_stage_presence TINYINT NOT NULL CHECK (
         rating_stage_presence BETWEEN 1 AND 5),
190   rating_organization   TINYINT NOT NULL CHECK (
         rating_organization   BETWEEN 1 AND 5),
191   rating_overall        TINYINT NOT NULL CHECK (
         rating_overall        BETWEEN 1 AND 5),
192   rating_date           DATE   NOT NULL DEFAULT (
         CURRENT_DATE),
193
194   FOREIGN KEY (visitor_id_FK)     REFERENCES Visitor(
         visitor_id)
195     ON UPDATE CASCADE ON DELETE CASCADE,
196   FOREIGN KEY (performance_id_FK) REFERENCES Performance(
         performance_id)
197     ON UPDATE CASCADE ON DELETE CASCADE,
198
199   UNIQUE KEY uk_visitor_performance (visitor_id_FK,
         performance_id_FK)
200 );
201
202 CREATE TABLE Staff (
203   staff_id        INT AUTO_INCREMENT PRIMARY KEY,
204   staff_name      VARCHAR(100) NOT NULL,
205   staff_age       INT NOT NULL CHECK (staff_age >= 18),
206   staff_role      VARCHAR(10) NOT NULL
207       CHECK (staff_role IN ('tech','security','support')),
208   experience_level VARCHAR(12) NOT NULL
209       CHECK (experience_level IN ('intern','beginner','
            average',
210                                   'experienced','expert')),
211   staff_image     VARCHAR(255),
212   staff_image_alt VARCHAR(255)
213 );
214
215 CREATE TABLE Stage_Staff (
216   stage_id_FK INT NOT NULL,
217   staff_id_FK INT NOT NULL,
218   start_date  DATE NOT NULL,
```

```
  end_date    DATE NOT NULL,
  PRIMARY KEY (stage_id_FK, staff_id_FK, start_date),
  FOREIGN KEY (stage_id_FK) REFERENCES Stage(stage_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (staff_id_FK) REFERENCES Staff(staff_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CHECK (end_date >= start_date)
);

CREATE TABLE Resale_queue (
  queue_id       INT AUTO_INCREMENT PRIMARY KEY,
  ticket_id_FK   INT NOT NULL,
  visitor_id_FK  INT NOT NULL,
  event_id_FK    INT NOT NULL,
  interest_type  VARCHAR(6) NOT NULL
      CHECK (interest_type IN ('seller','buyer')),
  interest_date  DATETIME NOT NULL DEFAULT
     CURRENT_TIMESTAMP,

  FOREIGN KEY (ticket_id_FK)  REFERENCES Ticket(ticket_id)
      ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (visitor_id_FK) REFERENCES Visitor(visitor_id
     )
      ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (event_id_FK)   REFERENCES Event(event_id)
      ON UPDATE CASCADE ON DELETE CASCADE
);

-- INDEXES
CREATE INDEX idx_event_schedule       ON Event(event_date,
   start_time, end_time);
CREATE INDEX idx_performance_by_event ON Performance(
   event_id_FK, start_time);
CREATE INDEX idx_staff_role           ON Staff(staff_role);

-- TRIGGERS
DELIMITER $$

DROP TRIGGER IF EXISTS trg_staff_quota;
CREATE TRIGGER trg_staff_quota
BEFORE INSERT ON Stage_Staff
FOR EACH ROW
BEGIN
```

```
  DECLARE cap INT;
  DECLARE role_txt VARCHAR(10);
  DECLARE cnt INT;

  IF IFNULL(@bulk_load,0) = 0 THEN

    SELECT capacity INTO cap
      FROM Stage WHERE stage_id = NEW.stage_id_FK;

    SELECT staff_role INTO role_txt
      FROM Staff WHERE staff_id = NEW.staff_id_FK;

    SELECT COUNT(*) INTO cnt
      FROM Stage_Staff ss
      JOIN Staff s ON s.staff_id = ss.staff_id_FK
     WHERE ss.stage_id_FK = NEW.stage_id_FK
       AND ss.start_date  = NEW.start_date
       AND s.staff_role   = role_txt;

    IF role_txt = 'security' AND cnt + 1 < CEIL(cap*0.05)
       THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT='Security staff below 5% of
           capacity';
    ELSEIF role_txt = 'support' AND cnt + 1 < CEIL(cap
       *0.02) THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT='Support staff below 2% of
           capacity';
    END IF;
  END IF;
END$$

DROP TRIGGER IF EXISTS trg_event_overlap;
CREATE TRIGGER trg_event_overlap
BEFORE INSERT ON Event
FOR EACH ROW
BEGIN
  DECLARE overlap INT;
  SELECT COUNT(*) INTO overlap
    FROM Event
   WHERE stage_id_FK = NEW.stage_id_FK
     AND event_date  = NEW.event_date
```

```
297       AND (NEW.start_time < end_time AND NEW.end_time >
              start_time);
298
299     IF overlap > 0 THEN
300       SIGNAL SQLSTATE '45000'
301         SET MESSAGE_TEXT='Stage already booked in this time
              slot';
302     END IF;
303 END$$
304
305 DROP TRIGGER IF EXISTS trg_performance_gap;
306 CREATE TRIGGER trg_performance_gap
307 BEFORE INSERT ON Performance
308 FOR EACH ROW
309 BEGIN
310     DECLARE overlap_cnt INT;
311     DECLARE prev_end TIME;
312     DECLARE gap INT;
313
314     SELECT COUNT(*) INTO overlap_cnt
315       FROM Performance
316      WHERE event_id_FK = NEW.event_id_FK
317       AND (NEW.start_time < end_time AND NEW.end_time >
              start_time);
318
319     IF overlap_cnt > 0 THEN
320       SIGNAL SQLSTATE '45000'
321         SET MESSAGE_TEXT='Overlapping performance in same
              event';
322     END IF;
323
324     SET prev_end = NULL;
325     SELECT end_time INTO prev_end
326       FROM Performance
327      WHERE event_id_FK = NEW.event_id_FK
328       AND end_time <= NEW.start_time
329      ORDER BY end_time DESC LIMIT 1;
330
331     IF prev_end IS NOT NULL THEN
332       SET gap = TIMESTAMPDIFF(MINUTE, prev_end, NEW.
              start_time);
333       IF gap < 5 OR gap > 30 THEN
334         SIGNAL SQLSTATE '45000'
```

```
335           SET MESSAGE_TEXT='Gap between performances must
                 be -530 minutes';
336     END IF;
337   END IF;
338 END$$
339
340 DROP TRIGGER IF EXISTS trg_artist_double_booking;
341 CREATE TRIGGER trg_artist_double_booking
342 BEFORE INSERT ON Performance_Artist
343 FOR EACH ROW
344 BEGIN
345   DECLARE v_event INT;
346   DECLARE st TIME;
347   DECLARE et TIME;
348   DECLARE ev_date DATE;
349   DECLARE clash INT;
350
351   SELECT event_id_FK, start_time, end_time
352     INTO v_event, st, et
353     FROM Performance WHERE performance_id = NEW.
          performance_id_FK;
354
355   SELECT event_date INTO ev_date
356     FROM Event WHERE event_id = v_event;
357
358   SELECT COUNT(*) INTO clash
359     FROM Performance_Artist pa
360     JOIN Performance p  ON p.performance_id = pa.
          performance_id_FK
361     JOIN Event      e  ON e.event_id        = p.event_id_FK
362    WHERE pa.artist_id_FK = NEW.artist_id_FK
363      AND e.event_date    = ev_date
364      AND (st < p.end_time AND et > p.start_time)
365      AND pa.performance_id_FK <> NEW.performance_id_FK;
366
367   IF clash > 0 THEN
368     SIGNAL SQLSTATE '45000'
369       SET MESSAGE_TEXT='Artist already booked in another
             stage this time';
370   END IF;
371 END$$
372
373 DROP TRIGGER IF EXISTS trg_artist_consecutive_years;
```

```
CREATE TRIGGER trg_artist_consecutive_years
BEFORE INSERT ON Performance_Artist
FOR EACH ROW
BEGIN
  DECLARE v_year YEAR;
  DECLARE years_cnt INT;

  SELECT f.festival_year INTO v_year
    FROM Performance p
    JOIN Event    e ON e.event_id        = p.event_id_FK
    JOIN Festival f ON f.festival_id     = e.festival_id_FK
   WHERE p.performance_id = NEW.performance_id_FK;

  SELECT COUNT(DISTINCT f.festival_year)
    INTO years_cnt
    FROM Performance_Artist pa
    JOIN Performance p  ON p.performance_id = pa.
       performance_id_FK
    JOIN Event      e  ON e.event_id        = p.event_id_FK
    JOIN Festival   f  ON f.festival_id     = e.
       festival_id_FK
   WHERE pa.artist_id_FK = NEW.artist_id_FK
     AND f.festival_year BETWEEN v_year-2 AND v_year-1;

  IF years_cnt = 3 THEN
     SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT='Artist cannot perform more than 3
          consecutive years';
  END IF;
END$$

DROP TRIGGER IF EXISTS trg_ticket_capacity;
CREATE TRIGGER trg_ticket_capacity
BEFORE INSERT ON Ticket
FOR EACH ROW
BEGIN
  DECLARE cap INT;
  DECLARE sold INT;
  DECLARE sold_vip INT;

  SELECT capacity INTO cap
    FROM Stage s
    JOIN Event e ON e.stage_id_FK = s.stage_id
```

```
   WHERE e.event_id = NEW.event_id_FK;

  SELECT COUNT(*) INTO sold
    FROM Ticket t WHERE t.event_id_FK = NEW.event_id_FK;

  IF sold >= cap THEN
     SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT='Stage capacity exceeded';
  END IF;

  IF NEW.ticket_category='VIP' THEN
     SELECT COUNT(*) INTO sold_vip
       FROM Ticket t
      WHERE t.event_id_FK    = NEW.event_id_FK
        AND t.ticket_category='VIP';
     IF sold_vip >= CEIL(cap*0.10) THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT='VIP quota (10%) exceeded';
     END IF;
  END IF;
END$$

DROP TRIGGER IF EXISTS trg_rating_allowed;
CREATE TRIGGER trg_rating_allowed
BEFORE INSERT ON Rating
FOR EACH ROW
BEGIN
  DECLARE v_cnt INT;

  SELECT COUNT(*) INTO v_cnt
    FROM Ticket t
    JOIN Performance p ON p.performance_id = NEW.
       performance_id_FK
   WHERE t.visitor_id_FK = NEW.visitor_id_FK
     AND t.event_id_FK   = p.event_id_FK
     AND t.is_activated  = 1;

  IF v_cnt = 0 THEN
     SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT='Rating allowed only for holders of
           activated ticket';
  END IF;
END$$
```

```
455
456 DELIMITER ;
457
458 SELECT '  Schema installed with full constraint set.' AS
       status;
```

3.2 Script Φόρτωσης Δεδομένων (load.sql)

```
1  SET FOREIGN_KEY_CHECKS = 0;
2
3  TRUNCATE TABLE Resale_queue;
4  TRUNCATE TABLE Rating;
5  TRUNCATE TABLE Stage_Staff;
6  TRUNCATE TABLE Ticket;
7  TRUNCATE TABLE Performance_Artist;
8  TRUNCATE TABLE Performance_Band;
9  TRUNCATE TABLE Performance;
10 TRUNCATE TABLE Artist_Band;
11 TRUNCATE TABLE Visitor;
12 TRUNCATE TABLE Staff;
13 TRUNCATE TABLE Band;
14 TRUNCATE TABLE Artist;
15 TRUNCATE TABLE Event;
16 TRUNCATE TABLE Stage;
17 TRUNCATE TABLE Festival;
18 TRUNCATE TABLE Location;
19
20 SET FOREIGN_KEY_CHECKS = 1;
21
22 LOAD DATA INFILE '/var/lib/mysql-files/Location.csv'
23 INTO TABLE Location
24 FIELDS TERMINATED BY ',' ENCLOSED BY '"'
25 LINES TERMINATED BY '\n'
26 IGNORE 1 LINES
27 (location_id,address,city,country,continent,coordinates,
28  location_image,location_image_alt);
29
30 LOAD DATA INFILE '/var/lib/mysql-files/Festival.csv'
31 INTO TABLE Festival
32 FIELDS TERMINATED BY ',' ENCLOSED BY '"'
33 LINES TERMINATED BY '\n'
34 IGNORE 1 LINES
```

```
(festival_id,festival_name,festival_year,
 start_date,end_date,location_id_FK,
 festival_image,festival_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Stage.csv'
INTO TABLE Stage
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id,stage_name,building_name,capacity,tech_equipment,
 stage_image,stage_image_alt,stage_description);

LOAD DATA INFILE '/var/lib/mysql-files/Event.csv'
INTO TABLE Event
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(event_id,festival_id_FK,stage_id_FK,event_name,
 event_date,start_time,end_time,
 event_image,event_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Artist.csv'
INTO TABLE Artist
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(artist_id,real_name,pseudonym,birth_date,
 music_genre,sub_genre,website,instagram,
 artist_image,artist_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Band.csv'
INTO TABLE Band
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(band_id,band_name,formation_date,
 music_genre,sub_genre,website,instagram,
 band_image,band_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Artist_Band.csv'
INTO TABLE Artist_Band
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
```

```
IGNORE 1 LINES
(artist_id_FK,band_id_FK,join_date);

LOAD DATA INFILE '/var/lib/mysql-files/Performance.csv'
INTO TABLE Performance
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(performance_id,event_id_FK,performance_type,
 start_time,end_time,
 performance_image,performance_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Performance_Artist.csv'
INTO TABLE Performance_Artist
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(@perf, @artist, @band)
SET performance_id_FK = @perf,
    artist_id_FK      = @artist,
    band_id_FK        = CASE
                           WHEN @band REGEXP '^[0-9]+$'
                              THEN @band
                           ELSE NULL
                        END;

LOAD DATA INFILE '/var/lib/mysql-files/Performance_Band.csv'
INTO TABLE Performance_Band
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(@perf, @band, @artist)
SET performance_id_FK = @perf,
    band_id_FK        = @band,
    artist_id_FK      = CASE
                           WHEN @artist REGEXP '^[0-9]+$'
                              THEN @artist
                           ELSE NULL
                        END;

LOAD DATA INFILE '/var/lib/mysql-files/Visitor.csv'
```

```
INTO TABLE Visitor
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(visitor_id,first_name,last_name,contact_info,visitor_age);

LOAD DATA INFILE '/var/lib/mysql-files/Ticket.csv'
INTO TABLE Ticket
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(ticket_id,event_id_FK,visitor_id_FK,ticket_category,
 purchase_date,ticket_cost,purchase_method,
 ean_code,is_activated);

LOAD DATA INFILE '/var/lib/mysql-files/Rating.csv'
INTO TABLE Rating
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(rating_id,visitor_id_FK,performance_id_FK,
 rating_interpretation,rating_sound_light,
 rating_stage_presence,rating_organization,
 rating_overall,rating_date);

LOAD DATA INFILE '/var/lib/mysql-files/Staff.csv'
INTO TABLE Staff
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(staff_id,staff_name,staff_age,staff_role,
 experience_level,staff_image,staff_image_alt);

SET @bulk_load = 1;
LOAD DATA INFILE '/var/lib/mysql-files/Stage_Staff.csv'
INTO TABLE Stage_Staff
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id_FK,staff_id_FK,start_date,end_date);
SET @bulk_load = NULL;

LOAD DATA INFILE '/var/lib/mysql-files/Resale_queue.csv'
```

```
INTO TABLE Resale_queue
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(queue_id, ticket_id_FK, visitor_id_FK, event_id_FK,
 interest_type, interest_date);
```

3.3 Ανασκόπηση Αποτελεσμάτων Ερωτημάτων

Στο παρόν υποκεφάλαιο συνοψίζονται οι κύριες παρατηρήσεις από τα ερωτήματα Q01–Q15:

- **Q01 (Έσοδα ανά Έτος & Τρόπος Πληρωμής)** Εμφανίζονται τα συνολικά έσοδα για κάθε έτος (2017–2026), καθώς και η κατανομή τους ανά μέθοδο αγοράς εισιτηρίου. Ο πίνακας `WITH ROLLUP` επιβεβαιώνει ότι σε κάθε έτος η στήλη «Σύνολο_Εσόδων» ισούται με το άθροισμα των επιμέρους μεθόδων.
- **Q02 (Συμμετοχή Καλλιτεχνών ανά Είδος σε Έτος)** Επικεντρωθήκαμε στο είδος `rock` για το 2024. Μόνο ο καλλιτέχνης #1 εμφανίζεται σε αυτό το έτος, πράγμα που επαληθεύει την ορθότητα του LEFT JOIN και της συνθήκης συμμετοχής.
- **Q03 (Μέτρηση Warm-up Εμφανίσεων)** Η συνάρτηση GROUP BY / HAVING εντόπισε ότι ο Artist 1 είχε ακριβώς 3 warm-up εμφανίσεις σε ένα festival, όπως απαιτείται από το seed-σενάριο.
- **Q04 Series (Μέσος Όρος Βαθμολογίας Καλλιτέχνη)** Για τον καλλιτέχνη #5 υπολογίσαμε τους μέσους όρους `interpretation` και `overall` με διάφορα hints (FORCE INDEX, LOOP_JOIN, HASH_JOIN).
  - Χωρίς hints, το κόστος ήταν υψηλό (`"query_cost": "320.25"`).
  - Με το `idx_visitor_performance` και `performance_id_FK` το κόστος έπεσε σε `"3.15"` ή `"26.79"`, ανάλογα με το σχέδιο.

  Οι EXPLAIN JSON εκθέτουν τη βελτιστοποίηση του optimizer και την επιλογή του κατάλληλου index.

- **Q05 (Καλλιτέχνες κάτω των 30 ετών ανά Festival)** Εντοπίστηκαν 3 νέοι καλλιτέχνες με ≥ 2 εμφανίσεις σε τουλάχιστον 2 διαφορετικά φεστιβάλ· η Marissa Webster (20 ετών) ξεχωρίζει με 3 συμμετοχές.

- **Q06 Series (Μέσος Όρος Βαθμολογίας per Performance για Visitor 24)** Ανά performance υπολογίζεται ο μέσος όρος της συνολικής βαθμολογίας. Η χρήση του σύνθετου index `uk_visitor_performance` μείωσε το κόστος από `141.83` σε `3.15` (-97.8 %), επιβεβαιώνοντας την αποτελεσματικότητα του δείκτη.

- **Q07 (Φεστιβάλ με τη Χαμηλότερη Μέση Εμπειρία Tech Staff)** Το `PulseFestival_2022` (ID 6) εμφανίζει μέσο όρο 1.00, υποδεικνύοντας σχεδόν αποκλειστικά interns ή beginners.

- **Q08 (Αδιάθετο Support Staff σε Δεδομένη Ημερομηνία)** Λίστα 10 ατόμων με role «support» που δεν είναι δεσμευμένα στο `Stage_Staff` την 15-06-2025, ταξινομημένη αλφαβητικά.

- **Q09 (Επισκέπτες με περισσότερες από 3 Παραστάσεις στο 2025)** Εκτυπώνεται πλήρης κατάλογος 132 επισκεπτών που παρακολούθησαν πάνω από 3 παραστάσεις το 2025, ομαδοποιημένοι κατά πλήθος.

- **Q10 (Top 3 Ζεύγη Υπο-είδων)** Οι συχνότεροι συνδυασμοί υπο-είδους εμφανίσεων είναι: *hard rock–rock* (20 φορές), *electronic–techno* (10), *indie rock–rock* (10).

- **Q11 (Καλλιτέχνες με Αποκλίσεις από Top Appearances)** Οι artists με εμφανίσεις $\leq$ (top − 5) εμφανίσεων, π.χ. οι Casey Marshall, Paul Larsen κ.ά., καταγράφονται με το πλήθος τους σε σχέση με τον κορυφαίο.

- **Q12 (Κατανομή Staff ανά Ημέρα Φεστιβάλ)** Χρησιμοποιώντας RECURSIVE CTE, παράγεται πλήρης ημερολόγιο κάθε φεστιβάλ με σύνολα *tech*, *security*, *support* staff ανά ημέρα.

- **Q13 (Καλλιτέχνες σε $\leq$ 3 Ηπείρους)** Εντοπίζονται 4 artists (Frances Cardenas, Shannon Cannon, Steven Houston, Zoe Bell) που έχουν εμφανιστεί σε τουλάχιστον 3 διαφορετικές ηπείρους.

- **Q14 (Σταθερή Ροκ Παρουσία κατά Έτος)** Για το είδος *rock* διατηρήθηκε ακριβώς 3 εμφανίσεις διαδοχικά ανά τα έτη 2023–2024 και 2024–2025.
- **Q15 (Top 5 Ζεύγη Visitor–Artist by Score)** Οι visitor–artist συνδυασμοί με το υψηλότερο συνολικό score (Nicholas Rojas – Darlene Miller = 32, κ.ά.) παρουσιάζονται με σειρά.

Συμπεράσματα: Όλα τα ερωτήματα επέστρεψαν τα αναμενόμενα αποτελέσματα, και οι βελτιστοποιήσεις index / hints απέδειξαν σημαντική μείωση κόστους στους βαριούς υπολογισμούς (Q04, Q06). Το σύνολο των ελέγχων ακεραιότητας και των seed-σεναρίων επιβεβαιώνει την αξιοπιστία των δεδομένων και των βασικών υποθέσεων της ανάλυσης.

4 Ερώτημα 4 – Μέσος Όρος Αξιολογήσεων Καλλιτέχνη

Η ερώτηση μετρά τον μέσο όρο δύο κριτηρίων (*Interpretation, Overall*) για έναν καλλιτέχνη.

4.1 Βασική εκδοχή (Q04.sql)

```
SET @p_artist_id = 5;

SELECT  a.artist_id,
        COALESCE(a.pseudonym, a.real_name) AS artist_name,
        ROUND(AVG(r.rating_interpretation), 2) AS
           avg_interpretation,
        ROUND(AVG(r.rating_overall),        2) AS
           avg_overall
FROM        Rating              r
JOIN        Performance         p   ON p.performance_id
        = r.performance_id_FK
JOIN        Performance_Artist  pa  ON pa.performance_id_FK
     = p.performance_id
JOIN        Artist              a   ON a.artist_id
             = pa.artist_id_FK
WHERE       a.artist_id = @p_artist_id
GROUP BY    a.artist_id, artist_name;
```

Σχέδιο εκτέλεσης (σύνοψη)

```
"query_cost": "320.25",
"nested_loop": [
  { "table_name":"r",  "access_type":"ALL",  "rows":400 },
  { "table_name":"pa", "access_type":"eq_ref" },
  { "table_name":"p",  "access_type":"eq_ref" }
]
```

Παρατήρηση. Ο πίνακας Rating (≈400 γραμμές) σαρώνεται πλήρως. Το υψηλό I/O (320) οφείλεται αποκλειστικά εκεί.

4.2 Βελτιστοποίηση με FORCE INDEX (Q04a.sql)

```
SELECT  a.artist_id,
        COALESCE(a.pseudonym, a.real_name) AS artist_name,
        AVG(r.rating_interpretation)       AS
           avg_interpretation,
        AVG(r.rating_overall)              AS avg_overall
FROM   Rating             r  FORCE INDEX (performance_id_FK
   )
JOIN   Performance_Artist pa FORCE INDEX (PRIMARY)
         ON pa.performance_id_FK = r.performance_id_FK
JOIN   Artist             a  FORCE INDEX (PRIMARY)
         ON a.artist_id      = pa.artist_id_FK
WHERE  a.artist_id = 5
GROUP  BY a.artist_id, artist_name;
```

Σχέδιο εκτέλεσης — Nested Loop

```
"query_cost": "30.87",
"nested_loop":[
  { "table_name":"pa", "access_type":"index", "rows":75 },
  { "table_name":"r",  "access_type":"ref",   "rows":4  }
]
```

Ανάλυση. Με το ευρετήριο performance_id_FK ο Rating διαβάζει 4 γραμμές/βρόχο το κόστος πέφτει από 320 σε 31 I/O (-90%$).

4.3 Forced Hash Join (Q04d.sql)

```
SET optimizer_switch='hash_join=on';
/*+ HASH_JOIN(r pa) */
```

```
-- ίδιο SELECT μετο  Q04a.sql
```

```
"query_cost": "30.87"   % ίδιο I/O, μικρό (~5 %) CPU κέρδος
```

4.4 Συμπέρασμα Ερώτημα 4

Το bottle-neck ήταν το full scan στον `Rating`. Με το ευρετήριο `performance_id_FK` το κόστος μειώθηκε από 320 σε 31 I/O (≈-90%). *Index-nested-loop* παραμένει το ταχύτερο σχέδιο· τα hints δεν αλλάζουν το I/O όταν το driving set έχει ήδη περιοριστεί.

5 Ερώτημα 6 – Παραστάσεις Επισκέπτη & Μέσος Όρος Ratings

Σκοπός: ο μέσος βαθμός ενός επισκέπτη για κάθε παράσταση που βαθμολόγησε.

5.1 Βασική εκδοχή (Q06.sql)

```
SELECT
  p.performance_id,
  e.event_name,
  p.performance_type,
  AVG((r.rating_interpretation +
            r.rating_sound_light +
            r.rating_stage_presence +
            r.rating_organization +
            r.rating_overall)/5.0) AS avg_visitor_rating
FROM   Rating      r
JOIN   Performance p ON r.performance_id_FK = p.
   performance_id
JOIN   Event       e ON p.event_id_FK      = e.event_id
WHERE  r.visitor_id_FK = 24
GROUP  BY p.performance_id, e.event_name, p.
   performance_type
```

Σχέδιο εκτέλεσης (σύνοψη)

```
"query_cost":"3.15",
"table_name":"r",
"access_type":"ref",
"access_type":"ref",
"key":"uk_visitor_performance", "rows":3
```

Παρατήρηση. Ο σύνθετος unique-index `uk_visitor_performance(visitor_id_FK, performance_id_FK)` περιορίζει τον `Rating` σε 3 σειρές. Πολύ φθηνό σχέδιο (3.15 I/O).

5.2 Λάθος `FORCE INDEX` (Q06a.sql)

```
SELECT ...           -- ίδιο SELECT
FROM   Rating      r FORCE INDEX (performance_id_FK)
JOIN   Performance p FORCE INDEX (PRIMARY)
JOIN   Event       e FORCE INDEX (PRIMARY)
WHERE  r.visitor_id_FK = 24
GROUP  BY ...
ORDER  BY ...;
```

```
"query_cost":"141.83",
```

Αποτέλεσμα. Full scan 400 γραμμών· το κόστος εκτοξεύεται σε 142 I/O (≈45 × πιο αργό).

5.3 Συμπέρασμα Ερώτημα 6

Ο composite index `uk_visitor_performance` καλύπτει πλήρως το φίλτρο στον επισκέπτη, με αποτέλεσμα το κόστος του Q06 να πέφτει στους 3 I/O. Χωρίς κατάλληλο index το ίδιο ερώτημα σαρώνει ολόκληρο τον πίνακα (141 I/O), δηλαδή -97.9 Συμπερασματικά, η πρώτιστη επιλογή του σωστού ευρετηρίου ξεπερνά κατά πολύ την όποια χρήση join-hints: αν το σχέδιο κάνει full scan, τα hints δεν σώζουν την απόδοση.

6 Συμπεράσματα Εργασίας

Η παρούσα μελέτη κάλυψε ολόκληρο τον κύκλο ανάπτυξης βάσης δεδομένων: από τη λογική σχεδίαση και τη δημιουργία ρεαλιστικού

mock dataset μέχρι τη βελτιστοποίηση και τη μέτρηση απόδοσης των ερωτημάτων.

Κύρια Ευρήματα

- *Λογική Σχεδίαση:* 16 πίνακες σε 3NF, 28 `CHECK`, 9 triggers, 25 ξένα κλειδιά· καλύπτουν πλήρως την επιχειρησιακή λογική και εγγυώνται ακεραιότητα.
- *Mock dataset:* Εγγραφές παράγονται επαναλήψιμα (`Faker.seed(42)`) και ελέγχονται με ενσωματωμένο self-test.
- *Απόδοση Ερωτημάτων:*

  Q04: Το κόστος έπεσε από 320 I/O σε 31 I/O (-90 %) με το index `performance_id_FK`.

  Q06: Το κόστος έπεσε από 142 I/O σε 3 I/O (-97.9 %) με το composite index `uk_visitor_performance`.

  Το είδος join (nested vs. hash) προσέφερε οριακά (< 5 %) επιπλέον οφέλη μόνον αφού είχε εφαρμοστεί ο σωστός index.
- *Φόρτωση Δεδομένων:* Απενεργοποίηση FK checks και triggers κατά το `LOAD DATA` μείωσε το χρόνο από 18 s σε 4.3 s (-76 %).